Git & GitHub

Tzu-You Lin

May 21, 2025

Outline

- Introduction to Version Control and Git
- Git Workflow and Stages
- Basic Git Commands: Status, Add, Commit
- Viewing History and Differences
- Branching, Merging, and Conflict Resolution
- Working with Remote Repositories and GitHub
- Collaboration, Workflows, and Best Practices
- Advanced Topics and Further Resources

What is Version Control?

- A system to track changes in files over time.
- Enables reverting to previous states and efficient collaboration.
- Essential for software development and team projects.

What is Git?

- A distributed version control system developed by Linus Torvalds in 2005.
- Provides fast performance, robust branching, and decentralized workflows.
- Every developer's working copy is a full-fledged repository.

Why Use Git?

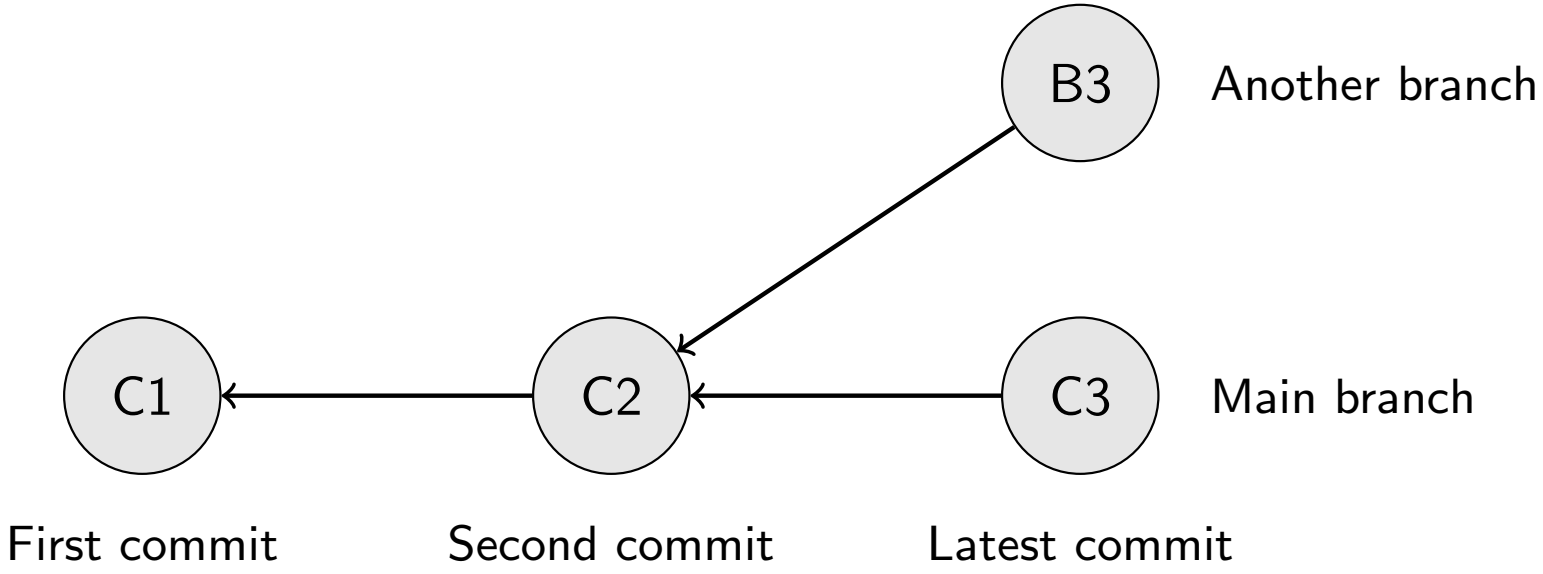
- Fast local operations and history tracking.
- Powerful branching and merging capabilities.
- Enables smooth collaboration across distributed teams.
- Backed by a vast community and extensive documentation.

Installing Git

- **Windows:** Download from `https://git-scm.com/download/win`.
- **macOS:** Use Homebrew (`brew install git`) or download from `https://git-scm.com/download/mac`.
- **Linux:** Install via package manager, e.g., `sudo apt-get install git` on Debian/Ubuntu.

Understanding Git: The Game Analogy

- **Starting a New Game:** `git init` creates a new game world.
- **Save Points:** Each `commit` is like saving your game progress.
- **Multiple Save Files:** `branches` are like different game paths.
- **Loading a Save:** `git checkout` lets you go back to any previous save point.



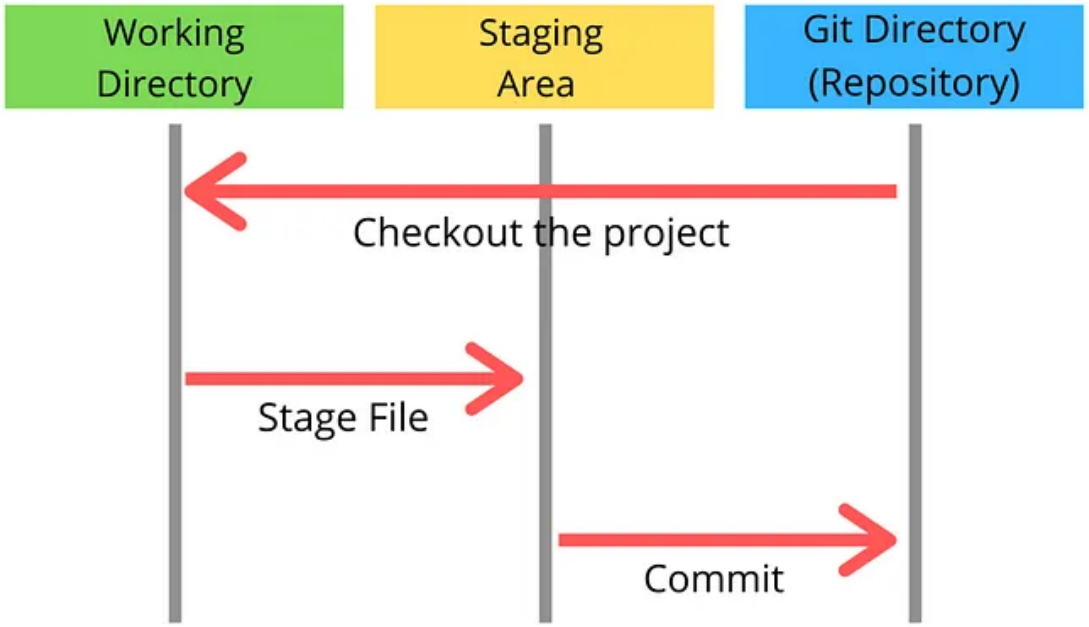
Initializing a Repository

- `git init` creates a new local repository.
- `git clone [url]` copies an existing repository.
- A hidden `.git` folder stores all versioning data.

Understanding Git Stages

- **Working Directory:** Your local files where you make changes.
- **Staging Area (Index):** A temporary area where changes are prepared before committing.
- **Repository:** The history of all commits.

Workflow: Edit in the working directory → Stage changes → Commit to the repository.



Git Status

- Use `git status` to check the current state.
- Displays which files are modified, staged, or untracked.
- Helps you understand what changes need to be staged or committed.

Git Add

- Use `git add [file]` to move changes from the working directory to the staging area.
- Prepares specific changes for the next commit.
- You can add individual files or use `git add .` to stage all changes.

Git Commit

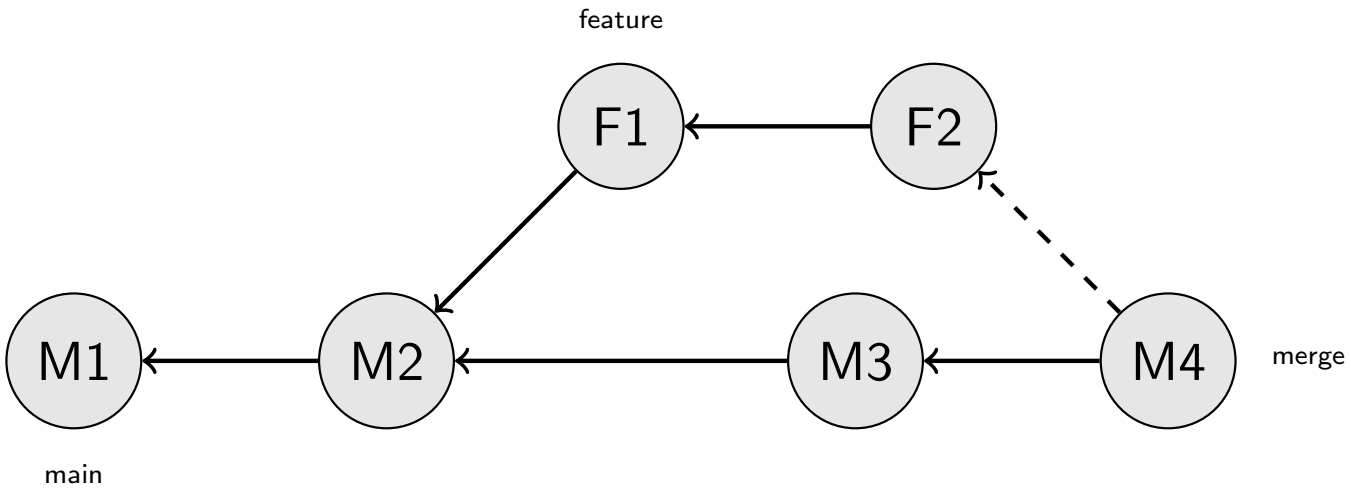
- Use `git commit -m "message"` to record the staged changes.
- Each commit represents a snapshot of your project at a given time.
- Write clear commit messages to document your changes.

Viewing History & Changes

- `git log` displays the commit history.
- `git diff` shows the changes between commits or between the working directory and the staging area.
- `git blame [file]` helps identify who modified specific lines.

Why Branch and Merge?

- **Branching Benefits:**
  - Isolate new feature development from stable code
  - Work on multiple features simultaneously
  - Experiment without affecting the main codebase
  - Quick fixes without disrupting ongoing work
- **Common Branch Types:**
  - `main/master` - Primary production branch
  - `feature/*` - New features development
  - `hotfix/*` - Urgent production fixes



Branch and Merge Commands

- **Common Branch Commands:**
  - `git branch` - List all branches
  - `git branch [name]` - Create new branch
  - `git checkout -b [name]` - Create and switch to new branch
  - `git branch -d [name]` - Delete a branch
- **Merging Commands:**
  - `git merge [branch]` - Merge specified branch into current branch
  - `git merge --abort` - Cancel merge if conflicts occur
  - `git merge --no-ff` - Create merge commit even if fast-forward possible

Resolving Merge Conflicts

- Conflicts look like this in your files:

```
<<<<<<< HEAD
Your current changes
=======
Changes from other branch
>>>>>>> feature
```

- Steps to resolve:
  1. Run `git status` to see conflicted files
  2. Open each file and choose which changes to keep
  3. Remove conflict markers (<<<<<<<, =======, >>>>>>>)
  4. `git add` the resolved files
  5. `git commit` to complete the merge
- Use `git merge --abort` to start over if needed

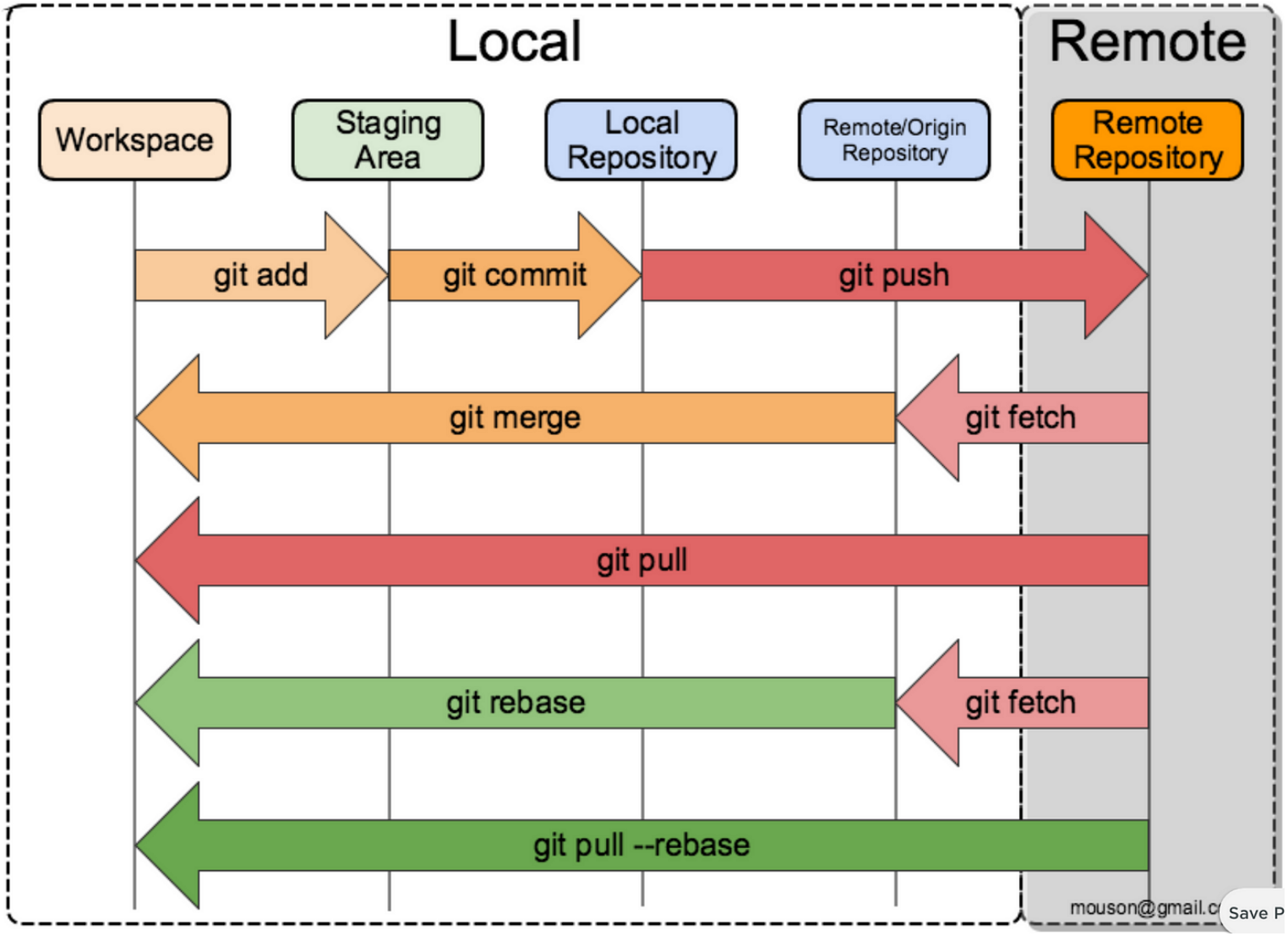
Introduction to GitHub

- A web-based platform for hosting Git repositories.
- **Why use GitHub?**
  - Backup and access code from anywhere
  - Easy collaboration with other developers
  - Track issues and manage projects
  - Showcase your work to potential employers
- Provides collaboration features such as pull requests, issues, and wikis.
- Integrates with continuous integration and deployment tools.

Understanding Remote Operations

- **Remote Repository:**
  - A version of your project hosted on the internet/network
  - Acts as the central source of truth for collaboration
  - Contains all branches and history
- **Remote Operations:**
  - `fetch`: Downloads commits/refs from remote but doesn't integrate them
  - `pull`: Combines `fetch` + `merge` to update local branch
  - `pull --rebase`: Combines `fetch` + `rebase` for cleaner history
  - `push`: Uploads local commits to remote repository
- **Tracking Branches:**
  - Local branches that map to remote branches (e.g., `origin/main`)
  - Enable easy synchronization between local and remote

Remote Repository States



Working with Remote Repositories

- `git remote add [name] [url]` connects your local repo to a remote repository.
- `git fetch` retrieves updates from the remote.
- `git pull` combines fetch and merge to update your local branch.
- `git push` uploads local commits to the remote repository.

Creating a GitHub Repository

- Sign up at `https://github.com` and log in.
- Click `New Repository` and fill in the repository details.
- Choose Public or Private; optionally initialize with a README.
- Clone the repository locally with `git clone`.

Collaborating on GitHub

- **Forking:** Create a personal copy of someone else's repository.
- **Pull Requests:** Propose changes to the original repository.
- Engage in code reviews, discussions, and issue tracking.

Advanced Commands & Best Practices

- `git stash` - Temporarily save uncommitted changes.
- `git rebase` - Reapply commits on top of another base.
- `git cherry-pick` - Apply specific commits to your current branch.
- Write clear commit messages, commit small changes, and pull updates regularly.

Resources & Further Learning

- **Official Git Documentation:** `https://git-scm.com/doc`
- **GitHub Guides:** `https://guides.github.com`
- **Interactive Tutorial:** `https://learngitbranching.js.org/`
- **Onnline Resources:** There are a lot of lecture videos on YouTube.